

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

**SOFTWARE ENGINEERING – LECTURES 09 – 11 – 12**

# System Design

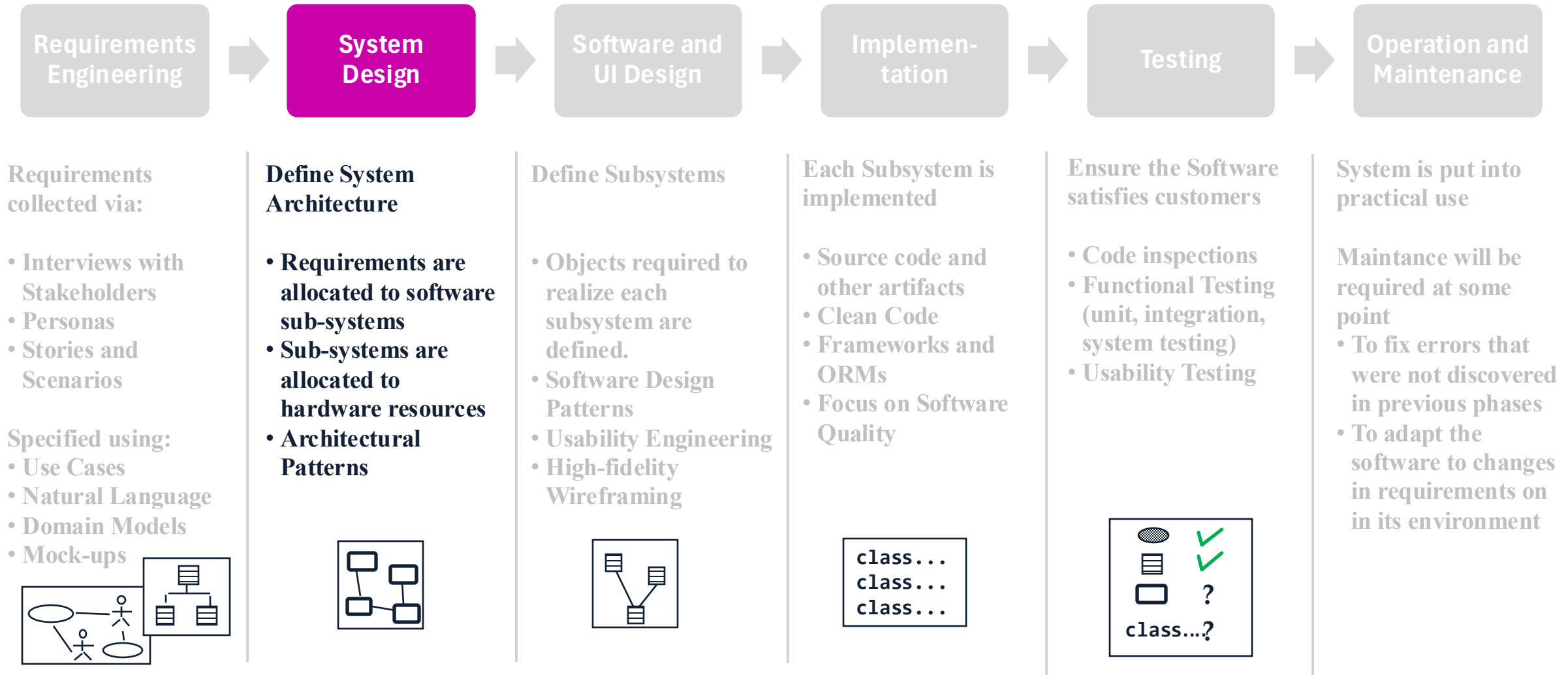
Prof. Bernardo Breve



# Lesson objectives

- Understanding System Design Activities
  - Structuring into Subsystems
    - Respecting System Objectives
  - Guidelines for Defining Artifacts
    - Concepts of Cohesion and Coupling between Artifacts
    - General Rules of Good Software Design

# THE WATERFALL SOFTWARE PROCESS MODEL



# System design

- System design is the set of tasks performed by the software engineer to transform the analysis model into the system design model.
- The ultimate goal of System and Object Design is to define a new document, written in technical/formal language, to be given to programmers so that they are able to implement the software described in the Software Requirements Document.

# Goal of System Design

1. Define the design objectives of the project
2. Define the system architecture, breaking it down into smaller subsystems
  - These can be implemented in parallel by individual teams
3. Select strategies for building the system, such as:
  - Hardware/software strategies
  - Strategies related to persistent data management
  - The overall control flow
  - Access control policies.
  - ...

# Goal of Object Design

- Given the system architecture, specify the implementation details of the smaller subsystems.
- Select the optimal strategies for implementation
  - Design Pattern

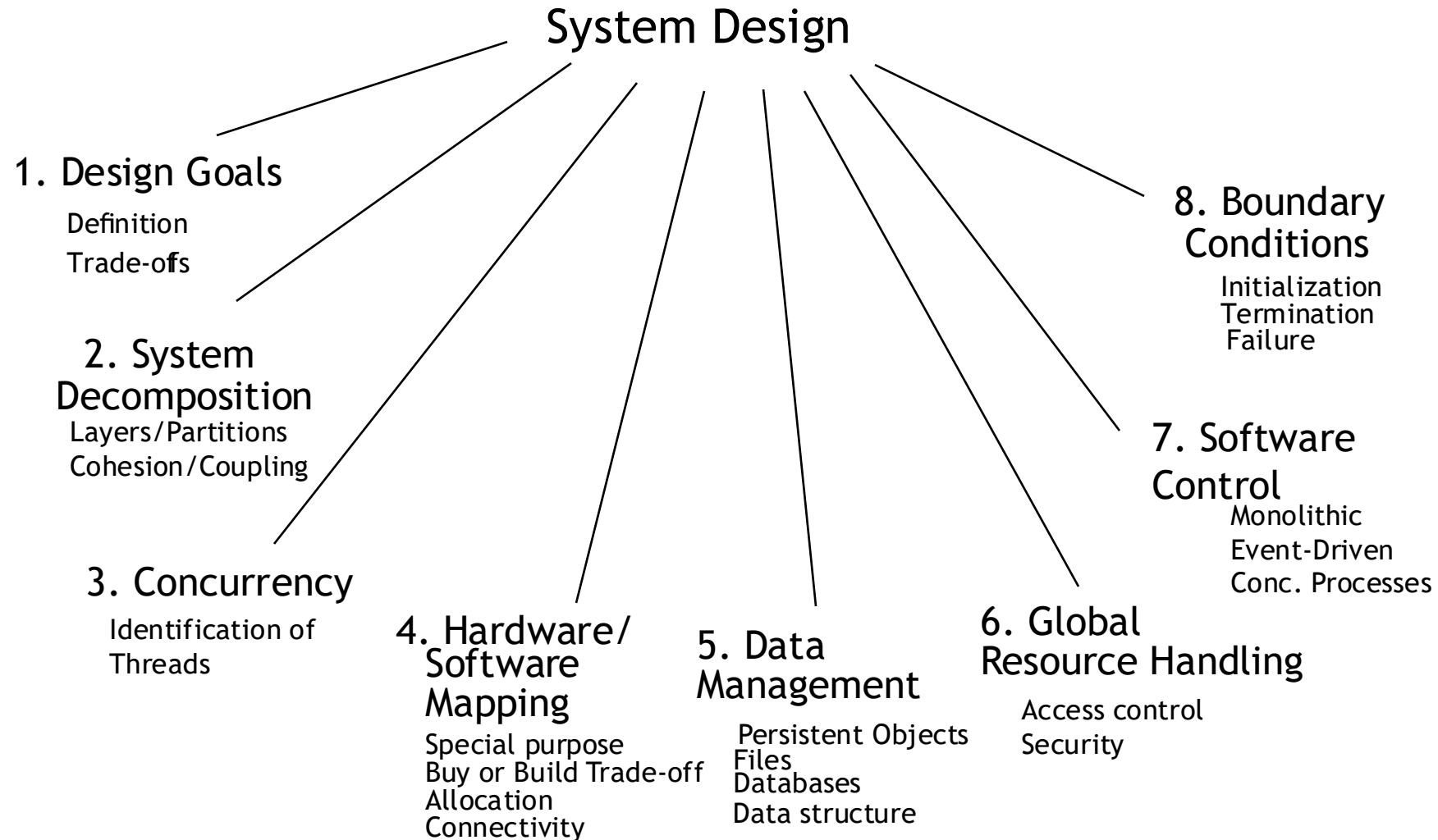
# System Design

- Radically change your point of view:
  - Until now, we have worked to interact with the customer
    - Language for laymen, not very formal
  - In system design, our interlocutors are the teams of programmers who will have to implement the system
    - Those who will read our documents will be IT experts
    - Language for technical experts

# System design activities

- System design consists of three macro-activities:
  1. Identify design objectives:
    - Developers identify which quality characteristics should be optimized and define the priorities for those characteristics
  2. Designing the decomposition of the system into subsystems:
    - Based on use cases and analysis models, developers break down the system into smaller parts. They use standard architectural styles
  3. Refine the breakdown into subsystems to meet design objectives:
    - The initial decomposition usually does not meet the design objectives. Developers refine it until the objectives are met

# System Design Steps



# Identify the qualitative objectives of the system

- This is the first step in system design
- Identify the **qualities** on which the system should focus
- Many design goals can be derived from non-functional requirements or the application domain, while others are provided by the customer
- It is important to formalize them explicitly, as every important design decision must be made according to the same set of criteria.

# Design criteria

- We can select design objectives from a long list of desirable qualities
- The criteria are organized into five groups:
  - Performance
    - Response time, throughput, memory requirements, ...
  - Reliability
    - Robustness, availability, fault tolerance, security, ...
  - Costs
    - Development, maintenance, management, ...
  - Maintenance
    - Extensibility, modifiability, adaptability, portability, ...
  - Usability

# Design Trade-offs

- When defining design objectives, often only a small subset of these criteria can be taken into account.
  - Es: For example, it is not realistic to develop software that is both secure and inexpensive.
- Developers must prioritize design objectives, while also taking into account managerial aspects such as schedule and budget constraints.

# Design Trade-offs (cont.)

- Examples:
  - Memory vs. Speed. If the software does not meet response time and throughput requirements, more memory must be used to speed up the system (e.g., caching, more redundancy). If the software does not meet memory requirements, it can be compressed at the expense of speed.
  - Release time vs. Functionality. If release times are tight, fewer features than requested may be released, but within the right time frame.
  - Release time vs. Quality. If release times are tight, the project manager can release the software on schedule with bugs and fix them later, or release the software late but with fewer bugs.
  - Release time vs. Staffing. It may be necessary to add resources to the project to increase productivity..

# Example

*Outline the eight steps of system design for a real application*



# Basic concepts of good design

*Code quality*



# System Design Concepts

- Many design criteria are closely dependent on the problem being addressed
  - Choosing the best balance between the constraints imposed is the task of the analyst, who operates based on their own experience/sensitivity
- However, there are generic design criteria that apply to any O-O software
  - When applied correctly, these criteria significantly improve the internal quality of the code without introducing any particular disadvantages

# Think ahead

- When designing a class, we should strive to think about what changes may be required in the future:

**“Designing to ANTICIPATE CHANGE”**

- Our initial choices must facilitate future evolution
  - This is one of the fundamental requirements, as well as a source of evaluation for the project!

# Decomposition

- To reduce the complexity of the solution, we break down the system into smaller parts, called subsystems.
- A subsystem typically corresponds to the part of the work that can be done independently by a single developer or a team of developers.
- By breaking down the system into relatively independent subsystems, project teams can work on individual subsystems with minimal communication overhead.
- In the case of complex subsystems, we further apply this principle and break them down into simpler subsystems.

# Decomposition

- $P$  = problem,
- $C$  = complexity of  $P$ ,
- $E$  = effort required to solve  $P$
- Given two problems  $P1$  and  $P2$ , if  $C(P1) > C(P2)$ , then  $E(P1) > E(P2)$ .
- But it has been shown that  $C(P1+P2) > C(P1) + C(P2)$ , and therefore  $E(P1+P2) > E(P1) + E(P2)$ .

# Modellazione di sottosistemi

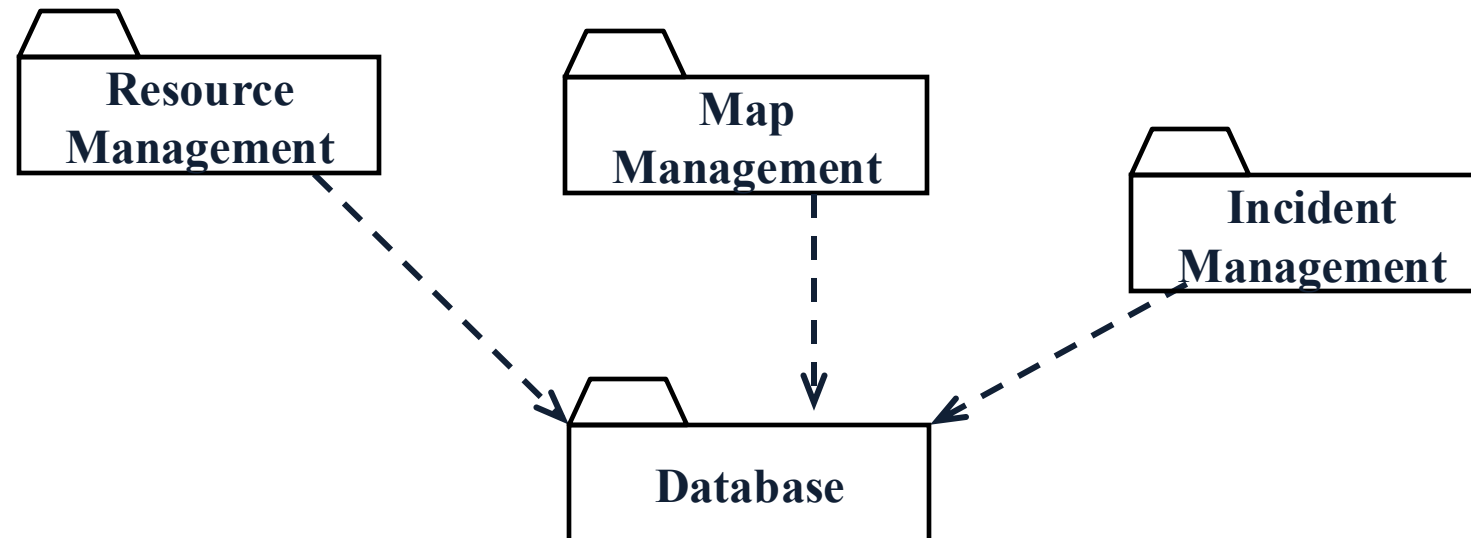
- Subsystem (UML: Package)
  - Collections of classes, associations, operations, and constraints that are related
- JAVA provides packages that are constructs for modeling subsystems
  - C++ / C# have the namespace construct to indicate the same concept

# Servizi e interfacce di sottosistemi

- A subsystem is characterized by the services it provides to other subsystems.
- The set of operations of a subsystem that are available to other subsystems forms the subsystem interface.
  - It includes the names of the operations, their parameters, their types, and their return values.
- System design focuses on defining the services provided by each subsystem in terms of:
  - Operations
  - Their parameters
  - Their high-level behavior

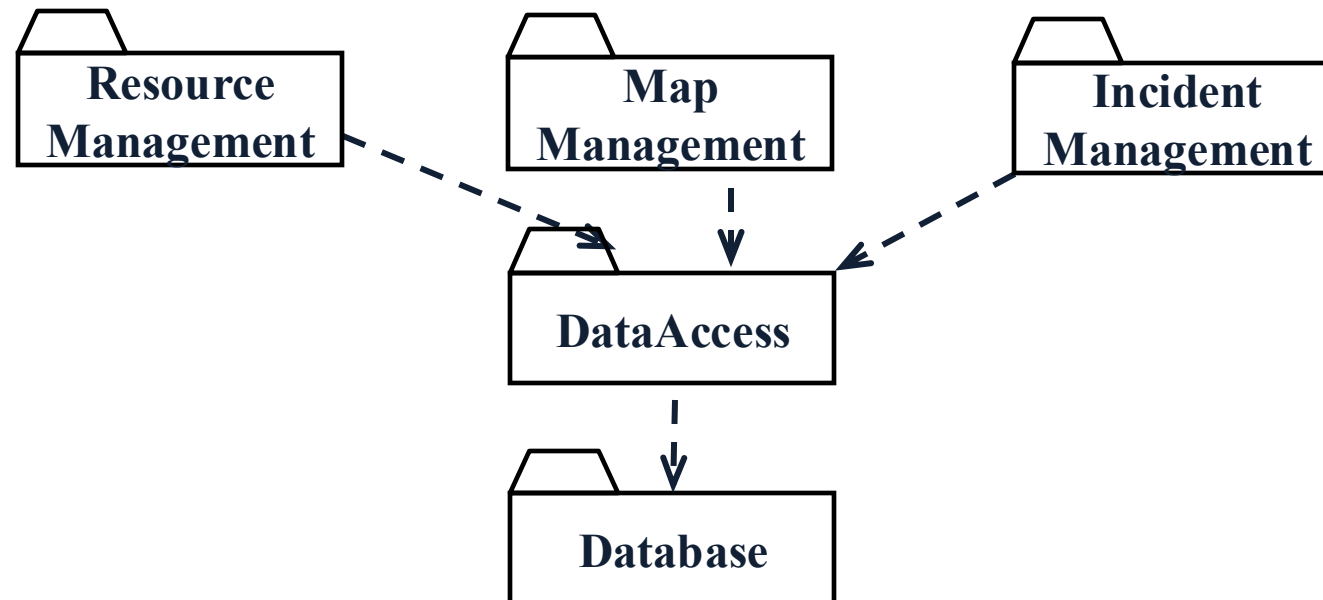
# Example of design choice: direct access to the database

- I cannot clearly specify the responsibilities of each module (Manage, Save?).
- All subsystems access the database directly, making them vulnerable to changes in the Database subsystem interface.
  - If the DBMS changes, I have to modify the code in all classes that access the DB!



# Example: Accessing the database through a storage subsystem

- Let's clarify responsibilities:
- I introduce the DataAccess subsystem that manages I/O from the database.
  - If the DBMS changes, I only need to modify the Storage subsystem!



# Observations

- In the example provided, we have reduced the relationships between the four subsystems, but...
- We have increased the complexity!
- With the aim of reducing relationships, there is a risk of adding levels of abstraction that consume development and processing time.

# Functional Independence

- The division into subsystems can be guided by two fundamental qualitative factors:
  - Cohesion
  - Coupling
- The goal is functional independence of the subsystems, which is maximized with high cohesion and low coupling.

# Rules of Good Software Design

*Cohesion*



# Cohesion

- Cohesion is a measure of how closely related and focused the responsibilities (or services offered) of a module are.
- If each unit is responsible for a single task, we say that that unit has high cohesion.
- High cohesion is a desirable property of code, as it allows you to:
  - Better understand the roles of a class
  - Reuse a class
  - Maintain a class
  - Limit and focus changes in the code
  - Use appropriate, effective, communicative names

# Granularity of Cohesion

- Method cohesion
  - A method should be responsible for only one well-defined task.
- Class cohesion
  - Each class should represent a single, well-defined concept.

# Cohesion - Single Responsibility Principle

- A class should have only ONE Responsibility
- A class should have only ONE reason to change
- A responsibility is “a reason for change.”
- The responsibilities of a class are axes of change.
  - If it has two responsibilities, they are coupled in the design, and so have to change together.
- If a class has more than one responsibility, it may become fragile when any of the requirements change

# Cohesion – Example

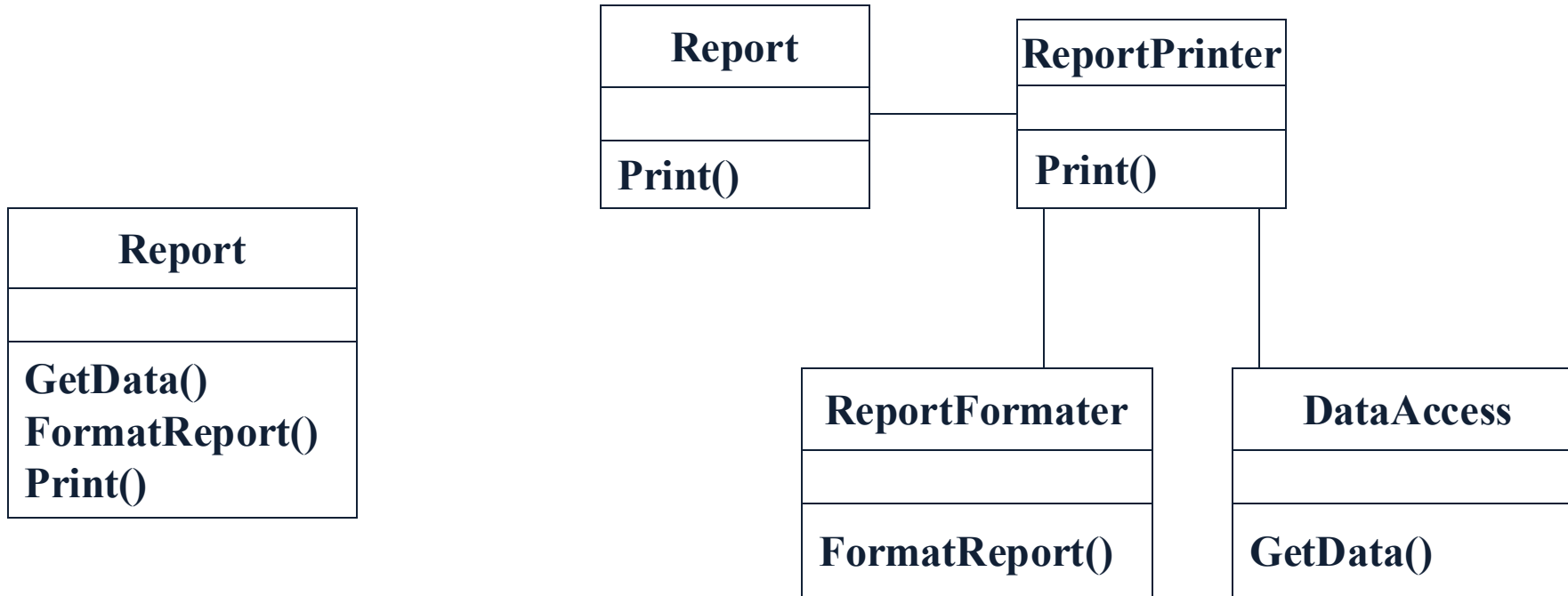
| Report                                 |
|----------------------------------------|
|                                        |
| GetData()<br>FormatReport()<br>Print() |

```
namespace Dimecasts.SOLID
{
    class Report
    {
        5 private IList<ReportDataElement> GetData()
        {
            Console.WriteLine("\nGetting Data...");
            return new List<ReportDataElement>() { new ReportDataElement() };
        }

        4 private void FormatReport()
        {
            Console.WriteLine("\nFormatting Report...");
        }

        6 public void Print()
        {
            GetData();
            FormatReport();
            Console.WriteLine("\nPrinting Report...");
        }
    }
}
```

# Cohesion – Example V2



# Cohesion – Example V2

```
public class ReportFormatter
{
    public void FormatReport ()
    {
        Console.WriteLine("\nFormatting Report...");
    }
}
```

```
public class DataAccess
{
    public IList<ReportDataElement> GetData ()
    {
        Console.WriteLine("\nGetting Data...");
        return new List<ReportDataElement> () { new ReportDataElement ()
    }
}
```

# Cohesion – Example V2

```
public class ReportPrinter
{
    public void Print()
    {
        DataAccess dataAccess = new DataAccess();
        ReportFormatter reportFormatter = new ReportFormatter();
        dataAccess.GetData();
        reportFormatter.FormatReport();
        Console.WriteLine("\nPrinting Report...");
    }
}

class Report
{
    public void Print()
    {
        ReportPrinter reportPrinter = new ReportPrinter();
        reportPrinter.Print();
    }
}
```

# Benefits of Cohesion

- Reuse - If all your classes follow the SRP, you are more likely to reuse some of them
- Clarity - Your code is cleaner as your classes don't do unexpected things
- Naming - As all your classes have a single responsibility, choosing a good name is easy
- Readability - Due to the clarity, better names and shorter files the readability is improved greatly.

# Cohesion

- Indicators of high cohesion (our goal)
  - A class has moderate responsibilities, limited to a single functional area
  - It collaborates with other classes to complete tasks
  - It has a limited number of methods, i.e., highly related functionalities
  - All methods seem to belong to the same set, with a similar overall goal
  - The class is easy to understand

# Rules of Good Software Design

*Coupling*



# Coupling

- Coupling is a measure of how strongly a class is connected to/has knowledge of/relies on other classes.
- Two or more classes are said to be coupled when it is impossible to reuse one without also reusing the other(s).
- If two classes depend closely and in many details on each other, we say that they are strongly coupled.
  - Otherwise, we say that they are weakly coupled.
- For quality code, we should aim for low coupling.

# Low coupling

- Low coupling is a desirable property of code, as it allows you to:
  - Understand the code of one class without reading the details of others
  - Modify one class without the changes affecting other classes
  - Easily reuse a class without having to import other classes
- Low coupling improves the maintainability of software

# Low coupling

- A certain level of coupling is inherent in the concept of message exchange in the O-O paradigm.
  - It is therefore necessary for the system to function.
  - It must be limited wherever possible.
- Guideline: define class responsibilities in such a way as to achieve low coupling.
  - The Law of Demeter (see following slides).

# Types of O-O Coupling

- Given two classes X and Y:
  1. X has an attribute of type Y, or
  2. X has a method that possesses an element (e.g., parameter, local variable, etc.) of type Y, or
  3. X is a subclass (possibly indirect) of Y, or
  4. X implements an interface of type Y
- Scenario 3 is the one that leads to the highest coupling.

# Example

```
class Traveler
{
    Car c=new Car();
    void startJourney()
    {
        c.move();
    }
}
```

```
class Car
{
    void move()
    {
        // logic...
    }
}
```

- What happens if we want to reuse our Traveler class with a means of transport other than Car?
- What happens if we want to change Car's methods?
- Traveler has an attribute of type Car, and therefore a strong coupling.

# Example (cont.)

```
class Traveler
{
    Vehicle v;
    public void setV(Vehicle v)
    {
        this.v = v;
    }

    void startJourney()
    {
        v.move();
    }
}

Interface Vehicle
{
    void move();
}
```

```
class Car implements Vehicle
{
    public void move()
    {
        // logic
    }
}
```

```
class Bike implements Vehicle
{
    public void move()
    {
        // logic
    }
}
```

# Explanation

- In the example, the introduction of a Vehicle interface has completely broken the coupling between Traveler and Car.
- Consequences:
  - Traveler is now coupled with an interface.
  - All implementations of the interface work with Traveler, without requiring code changes.
  - It will be possible to use Vehicle implementations in the future that have not yet been implemented today (anticipating change).
  - An external entity is needed to inject the actual implementation of the interface.
  - The complexity of the code has increased.

# Thinking about Interfaces

- Someone on the Java development team who understood interfaces decided to make them a big part of the Java SDK. There are hundreds of examples in the J2SE SDK of the correct way to use interfaces.
- Here are two key benefits derived from using interfaces:
  - An interface provides a means of setting a standard. It defines a contract that promotes reuse. If an object implements an interface then that object is promising to conform to a standard. An object that uses another object is called a consumer. An interface is a contract between an object and its consumer.
  - An interface also provides a level of abstraction that makes programs easier to understand. Interfaces allow developers to start talking about the general way that code behaves without having to get in to a lot of detailed specifics.

# The Paperboy example

```
public class Customer {  
    private String firstName;  
    private String lastName;  
    private Wallet myWallet;  
    public String getFirstName() {  
        return firstName;  
    }  
    public String getLastName() {  
        return lastName;  
    }  
    public Wallet getWallet() {  
        return myWallet;  
    }  
}
```

```
public class Wallet {  
    private float value;  
    public float getTotalMoney() {  
        return value;  
    }  
    public void setTotalMoney(float  
newValue) {  
        value = newValue;  
    }  
    public void addMoney(float deposit) {  
        value += deposit;  
    }  
    public void subtractMoney(float debit)  
{  
        value -= debit;  
    }  
}
```

# The Paperboy example

```
code from some method inside the Paperboy class...  
payment = 2.00; // "I want my two dollars!"
```

```
Wallet theWallet = myCustomer.getWallet();  
if (theWallet.getTotalMoney() > payment) {  
    theWallet.subtractMoney(payment);  
} else {  
    // come back later and get my money  
}
```

# Why Is This Bad?

- Let's translate what the code is actually doing into real-language:
  - Apparently, when the paperboy stops by and demands payment, the customer is just going to let the paperboy take the wallet out of his back pocket, and take out two bucks.
  - I don't know about you, but I rarely let someone handle my wallet. There are a number of 'real-world' problems with this, not to mention we are trusting the paperboy to be honest and just take out what he's owed.
  - If our future Wallet object holds credit cards, the paperboy has access to those too... but the basic problem is that “the paperboy is being exposed to more information than he needs to be”.
  - That's an important concept... The 'Paperboy' class now 'knows' that the customer has a wallet, and can manipulate it. When we compile the Paperboy class, it will need the Customer class and the Wallet class. These three classes are now 'tightly coupled'. If we change the Wallet class, we may have to make changes to both of the other classes.

# Why Is This Bad?

- There is another classic problem that this can create... What happens if the Customer's wallet has been stolen? Perhaps last night a Thief picked the pocket of our example, and someone else on our software development team decided a good way to model this would be by setting the wallet to null, like this:

```
victim.setWallet(null);
```

- This seems like a reasonable assumption... Neither the documentation nor the code enforces any mandatory value for wallet, and there is a certain 'elegance' about using null for this condition. But what happens to our Paperboy? Go look back at the code...
- The code assumes there will be a wallet! Our paperboy will get a runtime exception for calling a method on a null pointer.
- We could fix this by checking for 'null' on the wallet before we call any methods on it, but this starts to clutter the paperboy's code... our real-language description is becoming even worse...
- “If my customer has a wallet, then see how much he has... if he can pay me, take it”...

# Improving The Original Code

```
public class Customer {  
    private String firstName;  
    private String lastName;  
    private Wallet myWallet;  
    public String getFirstName() {  
        return firstName;  
    }  
    public String getLastName() {  
        return lastName;  
    }  
}
```

```
    public float getPayment(float bill) {  
        if (myWallet != null) {  
            if (myWallet.getTotalMoney() > bill)  
            {  
                theWallet.subtractMoney(payment);  
                return payment;  
            }  
        }  
    }  
}
```

The **Customer** no longer has a 'getWallet()' method, but it does have a getPayment() method.

# Improving the original code

```
// code from some method inside the Paperboy class...
    payment = 2.00;  // "I want my two dollars!"
    paidAmount = myCustomer.getPayment(payment);
    if (paidAmount == payment) {
        // say thank you and give customer a receipt
    } else {
        // come back later and get my money
    }
}
```

# Why is this better?

- That's a fair question... Someone who really liked the first bit of code could argue that we have just made the Customer a more complex object. That's a fair observation, and I'll address that when I talk about drawbacks, below.
- The first reason that this is better is because it better models the real world scenario... The Paperboy code is now 'asking' the customer for a payment. The paperboy does not have direct access to the wallet.
- The second reason that this is better is because the Wallet class can now change, and the paperboy is completely isolated from that change.
  - If the interface to Wallet were to change, the Customer would have to be updated, but that's it...
  - As long as the interface to Customer stays the same, none of the client's of Customer will care that he got a new Wallet.
  - Code will be more maintainable, because changes will not 'ripple' through a large project.

# Why is this better?

- The third, and probably most 'object-oriented' answer is that we are now free to change the implementation of 'getPayment()'. In the first example, we assumed that the Customer would have a wallet. This led to the null pointer exception we talked about. In the real world though, when the paper boy comes to the door, our Customer may actually get the two bucks from a jar of change, search between the cushions of his couch, or borrow it from his roommate.
- All of this is 'Business Logic', and is of no concern to the paper boy... All this could be implemented inside of the getPayment() method, and could change in the future, without any modification to the paper boy code.

# What Are The Drawbacks?

- Is the Customer REALLY becoming more complex? In our first example, the paperboy code using the Customer also had to know about and manipulate the Wallet. This is no longer the case...
- If Customer has become MORE complex, then why are the clients now LESS complex? All the same complexity still exists, it is just better contained within the scope of each object, and exposed in a healthy way.
- By containing this complexity, we get all the benefits discussed above. So yes, we can concede that Customer has gotten more complex. The truth is, it's complexity that existed before in the code, and it may have existed in several places. Now it occurs once, and can be maintained in the same place that the changes that may break it occur.

# Rules of Good Software Design

*Other rules*



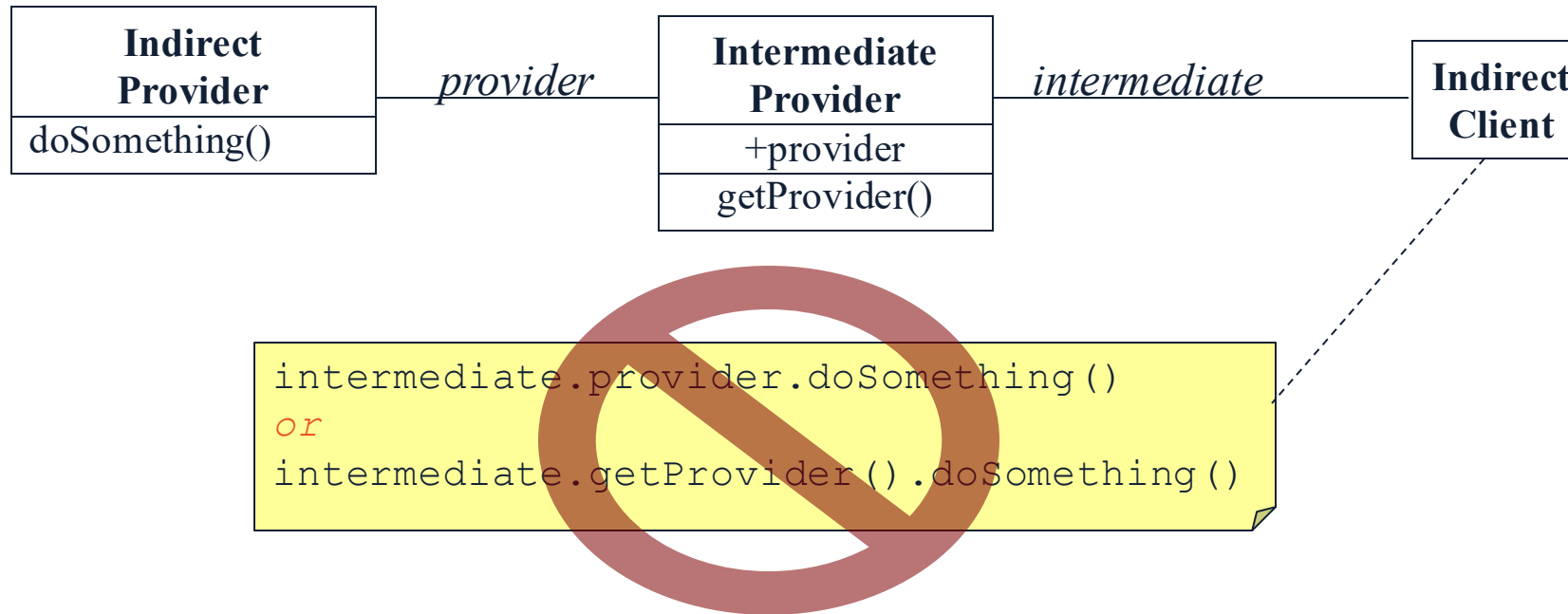
# The Law of Demeter

- This is another key guideline for achieving low coupling.
- It was used in the Demetra project, one of the first large O-O systems, developed at Boston University.
- Basic concept: Push information hiding to the maximum.
  - No situations such as: `Foo.getA().getB().getC()...`
  - The longer the call path traversed by the program, the more fragile it is to changes.

# The Law of Demeter

- A method should only send messages to:
  - The containing object (this)
  - An instance variable of this
  - A parameter of the method
  - An object created within the method

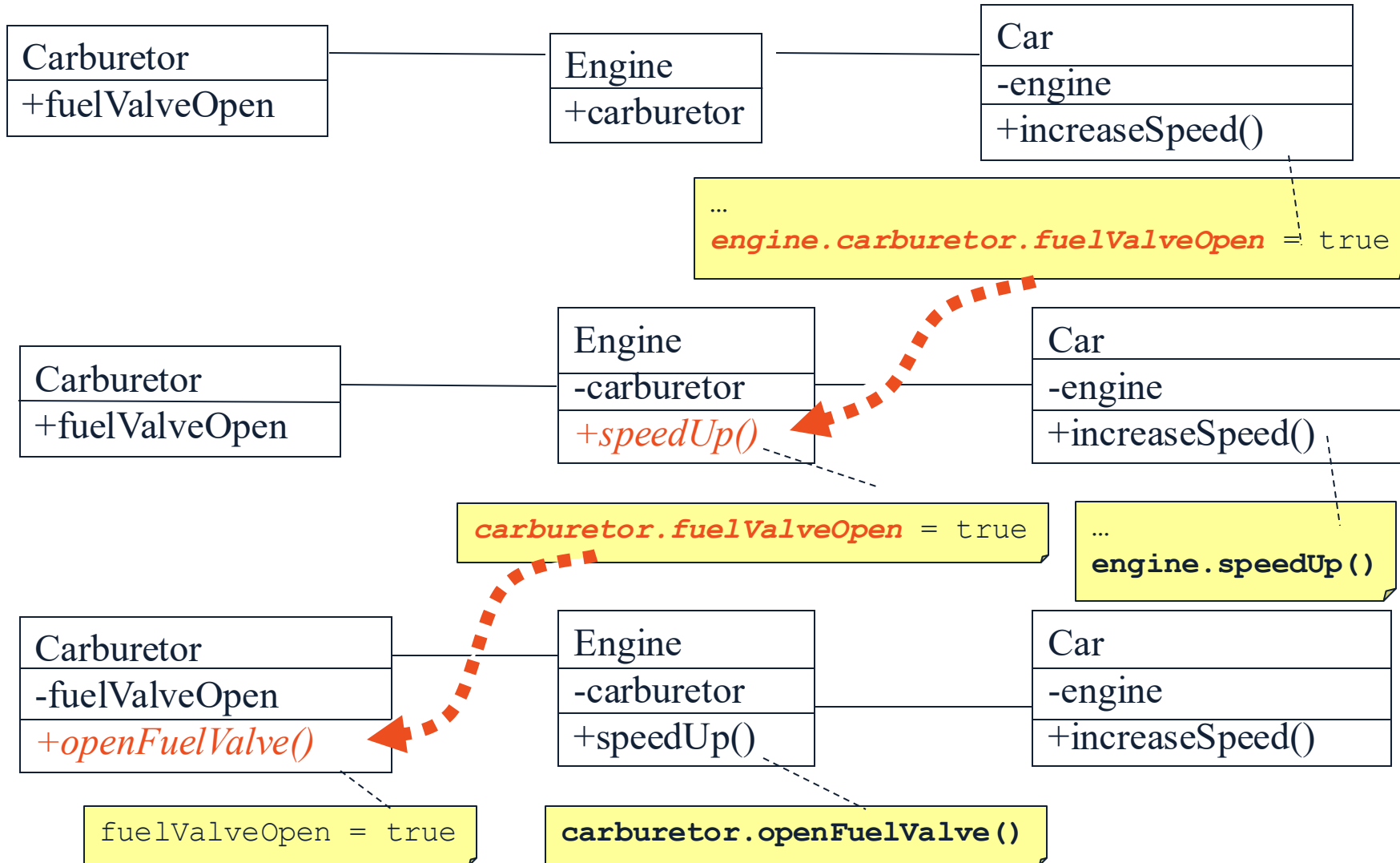
# The Law of Demeter, violated



**Law of Demeter:** A method "M" of an object "O" should invoke only the methods of the following kinds of objects.

1. *itself*
2. *its parameters*
3. *any object it creates /instantiates*
4. *its direct component objects*

# Eliminate Navigation Code



# Responsibility-driven Design



# Responsibility-driven design

- Think about system design in terms of:
  - Class
  - Responsibility
  - Collaboration
- Each class should have clearly defined responsibilities and roles → High Cohesion
- The class that owns the data should be responsible for processing it → Low Coupling
  - Each class should be responsible for managing its own data → Encapsulation
- It is facilitated by the use of CRC Cards.

# CRC Cards

- Introduced by Kent Beck and Ward Cunningham to teach O-O design
- A CRC card is a card associated with each class, representing the following concepts:
  - Class Name
  - Class Responsibilities
  - Class Collaborators
- Responsibilities: knowledge that the class maintains or services that the class provides
- Collaborator: another class whose knowledge or services must be exploited in order to fulfill the above responsibilities

# A CRC Card

- Associated with each class in the design class diagram

[illegible]

# Consequences

- CRC cards encourage you to achieve a design with clear responsibilities and control over the number of associated classes.
  - If I cannot clearly specify the responsibilities of a class, there is a design error.
  - If a class needs too many collaborating classes, there is a potential design error.

# Locate changes

- RDD design, with CRC Cards, leads to localization of changes
- When a change needs to be made, as few classes as possible should be involved
- The quality of your code is evident when the need to make changes arises